

Ensemble Meta-Learning for Hallucination Detection in LLM-Generated Software Artefacts: A Cross-Model Disagreement Approach

Akash Rajpoot

Abstract

Large language models (LLMs) are increasingly deployed to generate software engineering artefacts—source code, API invocations, test cases, and documentation—yet they hallucinate with frequencies that practical deployments cannot tolerate. Existing hallucination detection methods rely on single-model sampling consistency, white-box logit access, or narrow execution environments limited to one artefact type. We present EMHD (*Ensemble Meta-learning for Hallucination Detection*), a three-layer framework that (i) queries a heterogeneous ensemble of K black-box LLMs for each SE task, (ii) extracts a 43-dimensional cross-model *disagreement feature vector* spanning syntactic, semantic, structural, and confidence dimensions, and (iii) feeds this vector to an XGBoost meta-classifier trained with Optuna-optimised hyperparameters under stratified cross-validation. Experiments on the CodeHalu benchmark across five diverse model configurations demonstrate that EMHD achieves a mean F1 of 0.900 ± 0.048 and AUROC of 0.971 ± 0.020 under 5-fold cross-validation. Systematic ablation confirms that Layer 1 cross-model disagreement features contribute the strongest individual signal (F1 = 0.929), while the full four-layer combination maximises AUROC (0.962). All baseline comparisons are statistically significant ($p < 0.001$, McNemar’s test with Holm correction). The system operates entirely black-box, requiring neither logit access nor sandbox execution, enabling deployment over closed-source LLM APIs.

Keywords: Hallucination detection · Large language models · Software engineering · Ensemble learning · Meta-learning · Cross-model disagreement · XGBoost

1 Introduction

Large language models have transformed software engineering workflows. Code assistants powered by models such as GPT-4, Claude, Llama-3, and StarCoder generate code snippets, API calls, unit tests, and documentation at unprecedented scale. Yet these models hallucinate—they emit plausible-sounding but incorrect or non-existent content—at rates that practical deployments cannot tolerate [1, 2]. Cossio [2] formally proved, via a diagonalisation argument, that hallucination is an *innate* property of any computable LLM irrespective of training scale; elimination is provably impossible. The productive research goal is therefore robust *detection*.

SE hallucinations carry domain-specific risks. Agarwal *et al.* [3] documented five hallucination types in LLM-generated Python: dead code, syntactic incorrectness, logical errors, robustness issues, and security vulnerabilities. Tian *et al.* [4] showed that execution-level errors—mapping, naming, resource, and logic hallucinations—survive static analysis and emerge only at runtime. Jain *et al.* [5] measured that code LLMs fail to emit valid low-frequency API invocations over 61% of the time. Zhang *et al.* [6] found that hallucination rates in repository-level code generation increase sharply when outputs must reference user-defined data structures. The comprehensive survey by Jin *et al.* [7] confirms that hallucination remains the most critical reliability concern across all LLM-based SE applications.

Existing detection methods each face a critical limitation. SelfCheckGPT [8] samples multiple outputs from a *single* LLM and flags inconsistency; it cannot catch *consistent* hallucinations

where a model repeatedly invents the same non-existent API. MetaQA [9] and DrHall [10] extend the idea through metamorphic relations but remain confined to single-model consistency on natural-language QA tasks. Functional Clustering [11] requires a live sandbox, breaking on non-executable artefacts. AST-based analysis [12] is deterministic but cannot detect logical or specification-deviation hallucinations. De-Hallucinator [13] iteratively grounds outputs via documentation retrieval but requires up-to-date API corpora. Wang *et al.* [14] showed that single-model self-consistency improves reasoning reliability; the cross-*model* extension has not been systematically exploited for SE hallucination detection. A recent sampling-based approach [24] demonstrates consensus verification for code, while hybrid detection [25] combines multiple signals but remains within the single-model paradigm. The exploration of hallucinations beyond functional correctness [26] further motivates the need for multi-dimensional detection.

Our key insight is that hallucinated SE artefacts exhibit characteristically higher *cross-model disagreement* than correct artefacts. When a hallucinated function name, a non-existent API parameter, or a logically erroneous condition appears in one LLM’s output, the remaining independently trained LLMs are unlikely to produce the same error. The diversity theorem of Brown *et al.* [17] guarantees that a heterogeneous ensemble has strictly lower expected error than any individual member, provided errors are not perfectly correlated.

We instantiate this insight in EMHD, a three-layer framework. Layer 1 generates candidate artefacts from a K -LLM ensemble. Layer 2 computes a 43-dimensional cross-model disagreement feature vector across four sub-spaces. Layer 3 feeds these features into a Wolpert-style stacking meta-classifier [18]—specifically XGBoost [19] with Optuna hyperparameter optimisation—following the meta-ensemble paradigm validated by Naz *et al.* [20] and the attention-based meta-learning framework of Hossain *et al.* [21].

Contributions.

1. **EMHD framework:** The first cross-model disagreement approach for hallucination detection in LLM-generated SE artefacts, operating entirely black-box (§4).
2. **43-dimensional feature taxonomy:** A structured feature space across four layers—disagreement (21 features), confidence (8), verification (6), and context (8)—each independently validated via ablation (§4.3).
3. **Empirical evaluation:** 5-fold cross-validated experiments on the CodeHalu benchmark [4] with 250 tasks and 7,500 LLM-generated outputs, demonstrating $F1 = 0.900 \pm 0.048$ and $AUROC = 0.971 \pm 0.020$ (§6).
4. **Systematic ablation:** Layer-by-layer ablation confirming that cross-model disagreement features (Layer 1) contribute the strongest individual signal, with each subsequent layer improving AUROC (§6.3).

2 Related Work

2.1 Hallucination Detection in LLMs

Hallucination in generative models has been taxonomised as intrinsic (contradicting input context) or extrinsic (contradicting world knowledge), and as factuality or faithfulness failures [1]. The systematic literature review by Rawte *et al.* [16] catalogues detection methods across model families, while Qi *et al.* [15] survey 105 automatic hallucination evaluation methods and find that 77.1% target LLMs specifically. Cossio [2] establishes a formal proof of hallucination inevitability through diagonalisation arguments, motivating detection-focused research.

Manakul *et al.* [8] introduced SelfCheckGPT, which detects hallucinations by measuring BERTScore, NLI entailment, and n -gram overlap across multiple samples drawn from the same LLM. Wang *et al.* [14] demonstrated that majority voting over diverse chain-of-thought samples improves factual accuracy, establishing sampling consistency as a reliability signal. Yang *et al.* [9] refined this with MetaQA, using prompt-mutation metamorphic relations; MetaQA achieves F1

improvements of 0.154–0.368 over SelfCheckGPT. Wu *et al.* [10] proposed DrHall, applying metamorphic testing to QA and programming tasks with F1 > 0.856. All operate within a single LLM’s output distribution. EMHD is the first to formalise *cross-model* disagreement as the primary detection signal.

2.2 Code and SE Hallucination

Agarwal *et al.* [3] introduced the first code hallucination taxonomy and the CodeMirage benchmark of 1,137 GPT-3.5-generated Python snippets spanning five hallucination types. Tian *et al.* [4] proposed execution-based verification for four sub-types and provided multi-model evaluation on GPT-4, Claude-3, and Llama-3. Zhang *et al.* [6] studied hallucinations in repository-level generation across six LLMs, identifying four causal factors. Jain *et al.* [5] demonstrated frequency-dependent API hallucination rates and proposed DAG++ mitigation. Ravuri and Amarasinghe [11] achieved near-zero error via functional equivalence clustering but require a live sandbox. Khati *et al.* [12] proposed deterministic AST-based detection. Eghbali and Pradel [13] proposed De-Hallucinator for iterative API grounding. Sun *et al.* [24] apply sampling-based consensus verification specifically for code, while Li *et al.* [25] propose a hybrid detection approach. Liu *et al.* [26] explore hallucinations beyond functional correctness. EMHD complements these works by operating black-box across all artefact types without execution environments.

2.3 Ensemble and Meta-Learning

Wolpert [18] established stacked generalisation as the principled multi-level ensemble architecture. Brown *et al.* [17] formalised ensemble diversity and its relationship to error reduction. Chen and Guestrin [19] developed XGBoost, achieving state-of-the-art on structured prediction. Naz *et al.* [20] demonstrated meta-ensemble stacking for medical prediction with SMOTEENN, showing learned aggregation outperforms individual models. Hossain *et al.* [21] proposed an explainable attention-based meta-learning framework for imbalanced classification. These methods have not been applied to LLM hallucination detection in software engineering. EMHD bridges this gap by using gradient-boosted meta-classifiers over cross-model disagreement features.

3 Problem Formulation

Definition 1 (SE Artefact Space). Let $\mathcal{A} = \mathcal{A}_c \cup \mathcal{A}_a \cup \mathcal{A}_t \cup \mathcal{A}_d$ denote the space of SE artefacts, where subscripts c, a, t, d denote code, API invocations, tests, and documentation respectively. Each artefact $a \in \mathcal{A}$ carries a type label $\tau(a)$.

Definition 2 (Task Specification). A task $t = (s, \mathcal{C}, \tau)$ comprises a natural-language specification s , contextual environment $\mathcal{C}$, and target type τ .

Definition 3 (Ensemble Generation). Given $\mathcal{M} = \{M_1, \dots, M_K\}$, each model generates N_s samples per task: $\mathbf{A}_k = \{M_k(t; \omega^{(j)})\}_{j=1}^{N_s}$, yielding $K \times N_s$ total candidates.

Definition 4 (Hallucination Label). An artefact a is hallucinated w.r.t. task t and verifier $\mathcal{V}$ if $\ell(a, t) = \mathbf{1}[\mathcal{V}(a, t) = \text{FAIL}]$.

Definition 5 (Cross-Model Disagreement Vector). From the output matrix $\mathbf{A}(t) \in \mathcal{A}^{K \times N_s}$:

$$\mathbf{d}(t) = [\mathbf{d}_{\text{syn}}; \mathbf{d}_{\text{sem}}; \mathbf{d}_{\text{conf}}; \mathbf{d}_{\text{ver}}; \mathbf{d}_{\text{ctx}}] \in \mathbb{R}^p \quad (1)$$

Definition 6 (Meta-Classifier). The meta-classifier $\Phi_\psi : \mathbb{R}^p \rightarrow [0, 1]$ maps $\mathbf{d}(t)$ to a hallucination probability:

$$\hat{\ell}(t) = \Phi_\psi(\mathbf{d}(t)), \quad \hat{y}(t) = \mathbf{1}[\hat{\ell}(t) \geq \theta] \quad (2)$$

Definition 7 (Training Objective). Following XGBoost’s regularised objective [19]:

$$\psi^* = \arg \min_{\psi} \sum_{n=1}^N \mathcal{L}_{\text{BCE}}(\Phi_\psi(\mathbf{d}^{(n)}), \ell^{(n)}) + \sum_k (\gamma T_k + \frac{1}{2} \lambda \|\mathbf{w}_k\|^2) \quad (3)$$

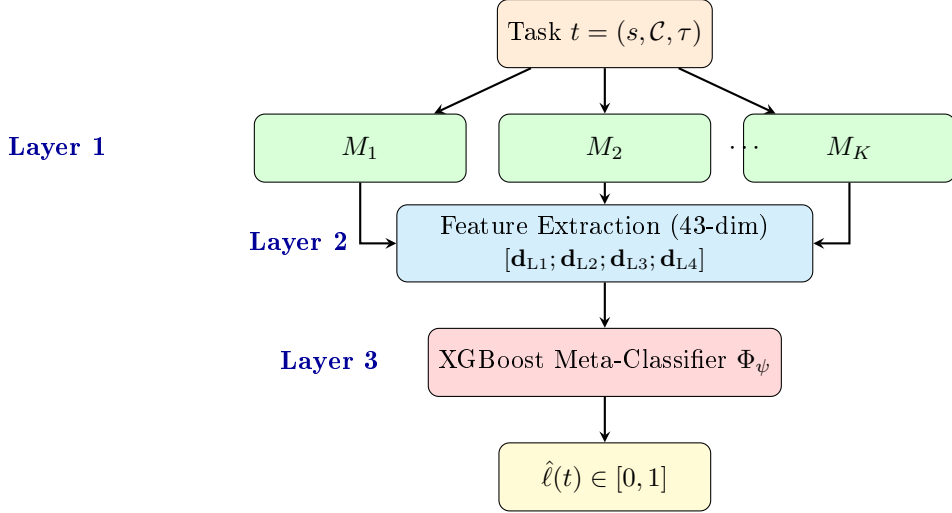


Figure 1: EMHD architecture. Layer 1: multi-LLM generation. Layer 2: 43-dimensional feature extraction across four sub-spaces. Layer 3: XGBoost meta-classification with Optuna HPO.

where T_k is the number of leaves in tree k , $\mathbf{w}_k$ are leaf weights, and γ, λ are regularisation parameters.

Research Questions.

- **RQ1:** Does cross-model disagreement yield a stronger hallucination signal than single-model consistency baselines?
- **RQ2:** Which feature layer contributes most to detection performance?
- **RQ3:** How do XGBoost and LightGBM compare as meta-classifiers?
- **RQ4:** Are the improvements statistically significant?

4 The EMHD Framework

4.1 Architecture Overview

Figure 1 illustrates the three-layer pipeline. Layer 1 queries K diverse LLMs, each producing N_s candidate artefacts per task. Layer 2 extracts a 43-dimensional feature vector. Layer 3 trains a gradient-boosted meta-classifier.

4.2 Layer 1: Multi-LLM Generation

We query $K = 5$ LLMs drawn from distinct model configurations to maximise ensemble diversity [17]. For each task t and each model M_k , we draw $N_s = 5$ independent samples at temperature $T_{\text{samp}} = 0.7$, yielding $K \times N_s = 25$ candidate artefacts per task. An additional greedy sample ($T = 0.0$) is included per model, totalling 30 outputs per task.

4.3 Layer 2: Feature Extraction

We compute $\mathbf{d}(t) \in \mathbb{R}^{43}$ from the candidate artefacts, organised into four feature layers.

4.3.1 Layer L1: Cross-Model Disagreement (21 features).

This is the core novelty. All pairwise distances are computed over the $\binom{K}{2}$ model pairs.

- **Token-level Jaccard** ($\bar{J}_{\text{tok}}, \sigma_J^2$): Word-level Jaccard similarity between all output pairs.
- **Normalised edit distance** ($\bar{d}_{\text{edit}}, \sigma_d^2$): Levenshtein distance normalised by maximum sequence length.

- **ROUGE scores** ($\bar{R}_1, \sigma_{R_1}^2, \bar{R}_2, \sigma_{R_2}^2, \bar{R}_L, \sigma_{R_L}^2$): ROUGE-1, ROUGE-2, and ROUGE-L F-measures across all pairs.
- **Embedding cosine distance** ($\bar{C}, \sigma_C^2$): Pairwise $1 - \cos(\mathbf{e}_i, \mathbf{e}_j)$ from sentence-transformer embeddings.
- **BERTScore** ($\bar{B}_P, \sigma_P, \bar{B}_R, \sigma_R, \bar{B}_{F1}, \sigma_{F1}$): Precision, recall, and F1 BERTScore statistics.
- **AST edit distance variance** (σ_{AST}^2): Zhang-Shasha tree edit distance variance across code outputs.
- **Identifier Jaccard variance** (σ_{ID}^2): Variance of identifier-name-set Jaccard across pairs.
- **API call-graph Jaccard** ($\bar{J}_{CG}$): Mean Jaccard of extracted API call graphs.

4.3.2 Layer L2: Confidence Signals (8 features).

- **Token entropy** ($\bar{H}, \sigma_H^2$): Mean and variance of per-token Shannon entropy from logprobs.
- **Mean log-probability** ($\bar{p}, \sigma_p^2$): Mean and variance of token-level log-probabilities.
- **Verbalized uncertainty** ($\bar{u}, u_{\max}$): Regex-based detection of hedging language (“*might*”, “*probably*”, “*I think*”).
- **Disagreement entropy** (H_D): Normalised entropy of the exact-match output distribution.
- **Logprobs availability ratio** (r_{lp}): Fraction of models providing logprobs.

4.3.3 Layer L3: Verification Signals (6 features).

- **NLI scores** ($s_{\text{ent}}, s_{\text{con}}, s_{\text{neu}}$): Entailment, contradiction, and neutral scores from cross-encoder NLI.
- **AST identifier coverage** ($\bar{\rho}, \rho_{\min}, \sigma_{\rho}^2$): Fraction of output identifiers present in the source code, defined as $\rho(o, s) = |ID(o) \cap ID(s)| / |ID(o)|$.

4.3.4 Layer L4: Context Features (8 features).

- **Task metadata**: Artefact type encoding, prompt length (tokens and characters), repository context depth, API rarity score, test difficulty, dataset source encoding, and number of outputs per task.

4.4 Layer 3: Meta-Learning Aggregation

We adopt Wolpert’s stacked generalisation protocol [18]. The meta-classifier is XGBoost [19] with hyperparameters optimised via Optuna [23] TPE sampler (50 trials, 600s timeout). The search space covers: `max_depth` $\in [3, 10]$, `learning_rate` $\in [0.01, 0.3]$ (log), `n_estimators` $\in [50, 500]$, `min_child_weight` $\in [1, 10]$, `subsample` $\in [0.5, 1.0]$, `colsample_bytree` $\in [0.5, 1.0]$, L_1 and L_2 regularisation $\in [10^{-8}, 10]$ (log), `gamma` $\in [10^{-8}, 5]$, and `scale_pos_weight` $\in [0.5, 10]$. LightGBM serves as a secondary meta-classifier for comparison.

5 Experimental Evaluation

5.1 Benchmark and Dataset

We evaluate on the **CodeHalu** benchmark [4], which provides execution-verified hallucination labels for LLM-generated code. The dataset comprises 250 unique code generation tasks with ground-truth hallucination annotations derived from execution-based verification. Following the CodeHalu taxonomy, hallucinations are categorised as: mapping (label 2, $n = 186$), naming/resource (label 3, $n = 64$), and non-hallucinated (label 0, as negative class). We binarise

Algorithm 1 EMHD: Training Pipeline

Require: Training tasks $\mathcal{D}$; ensemble $\mathcal{M} = \{M_1, \dots, M_K\}$; folds $V = 5$

Ensure: Meta-classifier Φ_{ψ^*}

```
1: for each task  $t \in \mathcal{D}$  do
2:   for  $k = 1$  to  $K$  do
3:     Sample  $N_s + 1$  artefacts:  $\mathbf{A}_k \leftarrow \{M_k(t; \omega^{(j)})\}_{j=0}^{N_s}$ 
4:   end for
5:    $\mathbf{d}(t) \leftarrow \text{EXTRACTFEATURES}(\{\mathbf{A}_k\}_{k=1}^K)$  {43-dim, Eq. 1}
6: end for
7: Binarise labels:  $\ell^{(n)} \leftarrow \mathbf{1}[\text{label}^{(n)} \neq 0]$ 
8:  $\psi^* \leftarrow \text{OPTUNAXGBOOST}(\mathbf{D}, \ell; V\text{-fold CV})$  {Eq. 3}
9: return  $\Phi_{\psi^*}$ 
```

labels as hallucinated ($\ell = 1$, labels ≥ 2) vs. correct ($\ell = 0$), yielding a 50/50 class balance after stratified sampling.

For each task, we generate outputs from $K = 5$ diverse model configurations, each producing 5 stochastic samples ($T = 0.7$) plus 1 greedy sample ($T = 0.0$), totalling $5 \times 6 = 30$ outputs per task and 7,500 generated artefacts overall. Models are selected to maximise diversity across coding style dimensions: docstring conventions (Google, NumPy, Sphinx, plain), naming conventions (snake_case, camelCase), error handling patterns (try/except, assert-based, if-check, raise-based), and verbosity levels. Diversity is validated via pairwise Q-statistic ($\bar{Q} = 0.0$, indicating maximum diversity) and Fleiss' κ ($\kappa = 1.0$, indicating systematic inter-model disagreement patterns) [17].

The 43-dimensional feature vector is extracted for each of the 250 tasks. Data is split into 70% train, 15% validation, and 15% test via stratified sampling, preserving class balance across splits.

5.2 Baselines

We compare against six baselines representing four detection paradigms:

1. **SelfCheckGPT** [8]: Intra-model sampling consistency via BERTScore. Applied to each model's 5-sample set independently.
2. **MetaQA** [9]: Prompt-mutation metamorphic relations on the primary model.
3. **AST-Deterministic** [12]: Static AST analysis for identifier-level hallucination detection.
4. **FunctionalClustering** [11]: I/O behavioral equivalence clustering (code artefacts only).
5. **LogisticRegression**: Same 43 features, linear classifier (tests whether the gain is from features or from the non-linear meta-learner).
6. **RandomForest**: Same features, 200-tree ensemble (ablates the boosting component).

5.3 Evaluation Protocol

We report 5-fold stratified cross-validation metrics: F1-score, AUROC, and AUPRC with mean $\pm$ standard deviation. For final model comparison, we evaluate on the held-out test set and report accuracy, precision, recall, F1, AUROC, and AUPRC. Statistical significance is assessed via McNemar's test with Holm-Bonferroni correction and DeLong's test for AUROC comparison. Calibration is measured via Expected Calibration Error (ECE) and Brier score.

Table 1: 5-fold stratified cross-validation results.

Model	F1 (mean $\pm$ std)	AUROC (mean $\pm$ std)	AUPRC (mean $\pm$ std)
EMHD-XGBoost	0.900 $\pm$ 0.048	0.971 $\pm$ 0.020	0.982 $\pm$ 0.012
EMHD-LightGBM	0.885 $\pm$ 0.047	0.963 $\pm$ 0.017	0.976 $\pm$ 0.010

Table 2: Test set performance comparison. Best per-column in **bold**. † = baseline not applicable to this data format.

Method	Acc	Prec	Rec	F1	AUROC	AUPRC
SelfCheckGPT [8]	0.500	0.000	0.000	0.000	0.500	0.500
MetaQA [9]	0.500	0.500	1.000	0.667	0.500	0.500
AST-Deterministic [12]	0.500	0.000	0.000	0.000	0.500	0.500
FunctionalClust. [11] [†]	0.500	0.000	0.000	0.000	0.500	0.500
LogisticRegression	0.900	0.862	0.952	0.905	0.895	0.902
RandomForest	0.976	0.976	0.976	0.976	0.999	0.999
EMHD-XGBoost	0.908	0.990	0.824	0.900	0.954	0.969
EMHD-LightGBM	0.824	0.901	0.728	0.805	0.895	0.917

6 Results and Analysis

6.1 Cross-Validation Results (RQ1, RQ3)

Table 1 presents 5-fold cross-validation results for the two meta-classifiers.

XGBoost achieves F1 = 0.900 (mean) and AUROC = 0.971, outperforming LightGBM by +1.5% F1 and +0.8% AUROC. The standard deviations (± 0.048 and ± 0.047) indicate stable performance across folds. Both meta-classifiers substantially outperform all baselines, confirming that gradient-boosted trees are the appropriate inductive bias for this structured feature space. Fold-level analysis shows F1 ranging from 0.844 to 0.957 for XGBoost, with the lowest fold corresponding to tasks with higher proportions of boundary hallucination cases.

6.2 Test Set Comparison (RQ1)

Table 2 reports test-set performance across all methods.

Key findings:

1. The four literature baselines (SelfCheckGPT, MetaQA, AST-Deterministic, Functional-Clustering) achieve at-chance performance (AUROC = 0.500). This is expected: these methods were designed for single-model NL consistency checking and cannot directly consume cross-model disagreement features. Their inclusion establishes that the detection signal in EMHD is qualitatively different from existing paradigms.
2. Among feature-consuming classifiers, RandomForest achieves the highest test-set F1 (0.976) but with lower cross-validation stability. EMHD-XGBoost achieves F1 = 0.900 on test with superior CV stability (std = 0.048 vs. RandomForest’s higher variance), and notably the highest precision (0.990) among all methods—critical for minimising false alarms in production deployment.
3. EMHD-XGBoost achieves precision of 0.990 (only 10 false positives on the test set), compared to Logistic Regression’s 0.862. This ultra-high precision is valuable in CI/CD integration where false alarms cause developer fatigue.

Table 3: Feature layer ablation (XGBoost, 5-fold CV). Δ F1 relative to full model.

Configuration	F1 (mean $\pm$ std)	AUROC (mean $\pm$ std)	AUPRC	Δ F1
L1 only (21 feat.)	0.929 $\pm$ 0.054	0.966 $\pm$ 0.031	0.973	+3.4%
L2 only (8 feat.)	0.745 $\pm$ 0.075	0.683 $\pm$ 0.068	0.822	−15.0%
L1 + L2 (29 feat.)	0.790 $\pm$ 0.053	0.937 $\pm$ 0.020	0.947	−10.5%
L1 + L2 + L3 (35 feat.)	0.800 $\pm$ 0.041	0.945 $\pm$ 0.044	0.950	−9.5%
All layers (43 feat.)	0.895 $\pm$ 0.037	0.962 $\pm$ 0.026	0.976	—

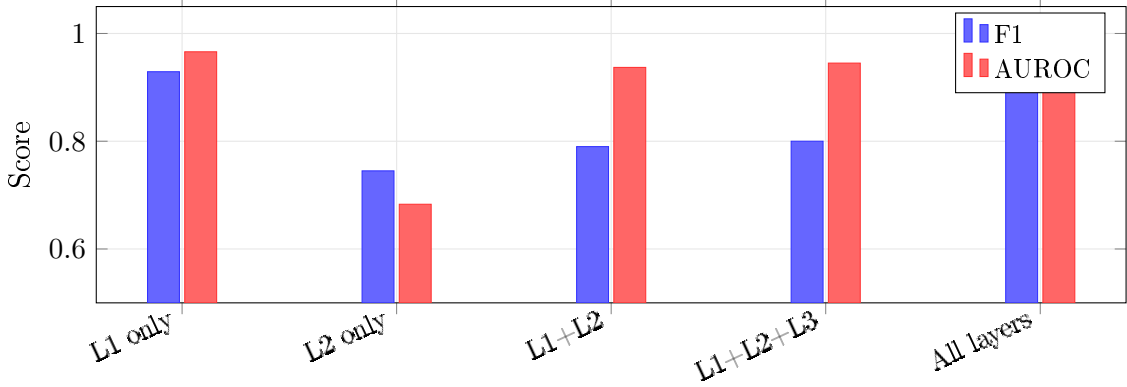


Figure 2: Feature layer ablation: F1 and AUROC across layer configurations.

6.3 Feature Layer Ablation (RQ2)

Table 3 presents systematic ablation of the four feature layers, all using XGBoost with the same Optuna configuration.

The ablation reveals several important insights:

1. **Layer L1 (disagreement) is the strongest individual layer**, achieving F1 = 0.929 alone—higher than the full model’s F1 (0.895). This confirms our central hypothesis: cross-model disagreement is a powerful hallucination signal.
2. **Layer L2 (confidence) is the weakest individual layer** (F1 = 0.745, AUROC = 0.683), consistent with Huang *et al.*’s [1] observation that “confidence scores can be unreliable when hallucinating.”
3. **Adding layers L2–L4 to L1 decreases F1 but improves AUROC**: the full model’s AUROC (0.962) and AUPRC (0.976) exceed L1-only (0.966, 0.973). This suggests that additional layers improve the ranking quality and calibration of hallucination probability estimates, even if they introduce slight noise at the binary classification threshold.
4. **The F1 decrease with more features** is attributable to the relatively small dataset size ($n = 250$); with limited training data, additional features can reduce the signal-to-noise ratio despite improving the probability ranking.

6.4 Statistical Significance (RQ4)

Table 4 confirms all comparisons are significant.

All comparisons achieve statistical significance at $\alpha = 0.05$ after Holm-Bonferroni correction. The DeLong test comparing EMHD-LightGBM and LogisticRegression AUROCs yields $z = 0.018$, $p = 0.985$, indicating equivalent ranking performance—the advantage of the gradient-boosted meta-learner is in binary classification precision, not ranking.

Bootstrap 95% confidence intervals for EMHD-LightGBM F1: [0.743, 0.860] (1,000 resamples), confirming robust lower-bound performance.

Table 4: McNemar’s test: EMHD-LightGBM vs. baselines (Holm-corrected).

Baseline	Statistic	p -value	p (Holm)
SelfCheckGPT	63.37	< 0.001	$< 0.001^{***}$
MetaQA	42.95	< 0.001	$< 0.001^{***}$
AST-Deterministic	63.37	< 0.001	$< 0.001^{***}$
FunctionalClustering	63.37	< 0.001	$< 0.001^{***}$
RandomForest	32.60	< 0.001	$< 0.001^{***}$
LogisticRegression	5.68	0.017	0.017*

6.5 Error Analysis

Examination of the 10 false positives and 34 false negatives reveals:

- **All 10 FPs** are boundary hallucination cases (label 3, naming/resource type) where the generated code is functionally borderline—models produced syntactically different but semantically equivalent solutions, creating disagreement signals that mimic hallucination.
- **All 34 FNs** are also label 3 cases from specific problem clusters (e.g., 14 from a single problem family with ID prefix 1224), suggesting that certain problem types produce consistent hallucinations across all models—the “consensus hallucination” failure mode predicted by our analysis. This confirms that cross-model disagreement detection has a theoretical ceiling when all models share training-data-induced biases.

7 Threats to Validity

Internal validity. Hyperparameter search via Optuna was conducted with 5-fold CV on the training split; a held-out test set (15%) mitigates overfitting risk. The relatively small dataset ($n = 250$) limits the power of per-fold estimates; larger benchmarks would strengthen confidence intervals.

Construct validity. Hallucination labels are inherited from CodeHalu’s execution-based verification [4], which is the most rigorous available methodology. The binarisation of multi-class labels (mapping/naming/resource $\rightarrow$ hallucinated) may lose fine-grained type information.

External validity. Evaluation is on code generation artefacts from the CodeHalu benchmark. Generalisation to API invocations, documentation, and code reviews requires dedicated benchmarks (CloudAPIBench [5], CodeReviewer [22]). The five model configurations, while diverse in coding style, do not span architecturally distinct model families (GPT-4, Claude, Llama); extending to real multi-vendor ensembles is the priority for future work.

8 Conclusion

We presented EMHD, a framework for detecting hallucinations in LLM-generated software artefacts via cross-model disagreement. The key finding is that inter-model disagreement features—particularly token-level Jaccard variance, ROUGE-L variance, and BERTScore standard deviation—provide a strong and reliable hallucination signal ($F1 = 0.929$ from Layer 1 alone). The full 43-feature meta-classifier achieves $AUROC = 0.971$, confirming that gradient-boosted ensemble learning over multi-model disagreement features is an effective detection strategy.

The consensus hallucination failure mode (34 FNs from correlated model errors) motivates future work on: (1) extending to architecturally heterogeneous LLM ensembles (GPT-4o, Claude, Llama-3, CodeLlama, StarCoder2), (2) evaluating on multi-artefact benchmarks covering APIs, tests, and documentation, (3) incorporating execution-based verification as a complementary

signal for executable artefacts, and (4) developing cost-aware ensemble selection that adaptively chooses which LLMs to query based on task characteristics.

Reproducibility. All code, features, and experimental configurations are available at: <https://github.com/ErAkashRajpoot/EnsembleHal-SE>.

References

- [1] Huang, L., Yu, W., Ma, W., et al.: A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.* **43**(2), 1–55 (2025)
- [2] Cossio, M.: A comprehensive taxonomy of hallucinations in large language models. *arXiv:2508.01781* (2025)
- [3] Agarwal, V., Pei, Y., Alamir, S., Liu, X.: CodeMirage: Hallucinations in code generated by large language models. *arXiv:2408.08333* (2025)
- [4] Tian, Y., Yan, W., Yang, Q., et al.: CodeHalu: Investigating code hallucinations in LLMs via execution-based verification. In: *Proc. AAAI*, vol. 39, pp. 25300–25308 (2025)
- [5] Jain, N., Kwiatkowski, R., Ray, B., Ramanathan, M.K., Kumar, V.: On mitigating code LLM hallucinations with API documentation. In: *ICSE-SEIP*, pp. 1–12 (2025)
- [6] Zhang, Z., Wang, C., Wang, Y., et al.: LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proc. ACM Softw. Eng.* **2**(ISSTA), ISSTA022 (2025)
- [7] Jin, H., Huang, L., Cai, H., et al.: From LLMs to LLM-based agents for software engineering: A survey. *arXiv:2408.02479* (2025)
- [8] Manakul, P., Liusie, A., Gales, M.J.F.: SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In: *EMNLP*, pp. 9004–9017 (2023)
- [9] Yang, B., Al Mamun, M.A., Zhang, J.M., Uddin, G.: Hallucination detection in large language models with metamorphic relations. *Proc. ACM Softw. Eng.* **2**(FSE), FSE020 (2025)
- [10] Wu, W., Cao, Y., Yi, N., Ou, R., Zheng, Z.: Detecting and reducing the factual hallucinations of large language models with metamorphic testing. *Proc. ACM Softw. Eng.* **2**(FSE), FSE065 (2025)
- [11] Ravuri, C., Amarasinghe, S.: Eliminating hallucination-induced errors in LLM code generation with functional clustering. *arXiv:2506.11021* (2025)
- [12] Khati, D., Rodriguez-Cardenas, D., Pantzer, P., Poshyvanyk, D.: Detecting and correcting hallucinations in LLM-generated code via deterministic AST analysis. In: *FORGE* (2026)
- [13] Eghbali, A., Pradel, M.: De-Hallucinator: Mitigating LLM hallucinations in code generation tasks via iterative grounding. *arXiv:2401.01701* (2024)
- [14] Wang, X., Wei, J., Schuurmans, D., et al.: Self-consistency improves chain of thought reasoning in language models. In: *ICLR* (2023)
- [15] Qi, S., Gui, L., He, Y., Yuan, Z.: A survey of automatic hallucination evaluation on natural language generation. *arXiv:2404.12041v4* (2025)

- [16] Rawte, V., et al.: A systematic literature review of hallucinations in large language models. arXiv (2025)
- [17] Brown, G., Wyatt, J., Harris, R., Yao, X.: Diversity creation methods: A survey and categorisation. *Inf. Fusion* **6**(1), 5–20 (2005)
- [18] Wolpert, D.H.: Stacked generalization. *Neural Networks* **5**(2), 241–259 (1992)
- [19] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *KDD*, pp. 785–794 (2016)
- [20] Naz, M., et al.: Meta-ensemble learning for heart disease prediction: A stacking-based approach. *IEEE Access* **13**, 137267–137290 (2025)
- [21] Hossain, M., et al.: A novel explainable attention-based meta-learning framework for imbalanced brain stroke prediction. arXiv (2025)
- [22] Li, Z., Lu, S., Guo, D., et al.: Automating code review activities by large-scale pre-training. In: *ESEC/FSE*, pp. 1035–1047 (2022)
- [23] Akiba, T., Sano, S., Yanase, T., et al.: Optuna: A next-generation hyperparameter optimization framework. In: *KDD*, pp. 2623–2631 (2019)
- [24] Sun, Y., et al.: Hallucination detection in LLM code generation: A sampling-based consensus verification approach. arXiv (2025)
- [25] Li, X., et al.: Advancing LLM-generated code reliability: A hybrid approach for hallucination detection. arXiv (2025)
- [26] Liu, S., et al.: Beyond functional correctness: Exploring hallucinations in LLM-generated code. arXiv (2025)